

Lesson 10 — DAX Solutions & Explanations

1) What does **FILTER(Sales, Sales[Amount] > 1000)** return?

FILTER(Sales, Sales[Amount] > 1000) returns a table containing all rows from the Sales table that meet the boolean condition Sales[Amount] > 1000. It's a table, not a scalar value.

2) Measure: High Sales (using FILTER)

DAX:

```
High Sales = CALCULATE( SUM(Sales[Amount]), FILTER(Sales, Sales[Amount] > 1000) )
```

Note: This works but can be optimized by using a boolean filter directly inside CALCULATE (see later).

3) How does **ALLEXCEPT(Sales, Sales[Region])** differ from **ALL(Sales)**?

ALLEXCEPT(Sales, Sales[Region]) removes all filters on the Sales table except any filters on Sales[Region].

ALL(Sales) removes all filters on the entire Sales table (ignores all columns).

4) SWITCH to categorize Amount

DAX:

```
Amount Category = SWITCH( TRUE(), Sales[Amount] > 1000, "High", Sales[Amount] >= 500 &&  
Sales[Amount] <= 1000, "Medium", "Low" )
```

5) What is the purpose of **ALLSELECTED**?

ALLSELECTED returns the set of values for the specified columns or table after applying the outermost filters coming from visuals, slicers, and the current query, but preserves the user selection in slicers. It's commonly used to compute context-aware percentages and running totals that respect slicer selections but ignore filters applied by the current visual.

6) Measure: Regional Sales % (each sale's contribution to its region's total)

DAX:

```
Regional Sales % = DIVIDE( SUM(Sales[Amount]), CALCULATE( SUM(Sales[Amount]), ALLEXCEPT(Sales,  
Sales[Region]) ) ) )
```

7) Dynamic measure: Toggle between SUM, AVERAGE, COUNT of Amount

DAX:

```
Amount Aggregation = VAR Mode = SELECTEDVALUE(ViewMode[Option], "Sum") RETURN SWITCH( Mode,  
"Sum", SUM(Sales[Amount]), "Average", AVERAGE(Sales[Amount]), "Count", COUNTROWS(Sales), BLANK()  
)
```

Note: ViewMode is a disconnected table with values {"Sum","Average","Count"} used as a slicer.

8) Excluding 'Furniture' category inside CALCULATE

DAX:

```
Sales Excluding Furniture = CALCULATE( SUM(Sales[Amount]), FILTER(Products, Products[Category] <>  
"Furniture") ) )
```

Better alternative: use a boolean filter that references the lookup table, e.g., Products[Category] <> "Furniture" inside CALCULATE directly when possible.

9) Why **ALLSELECTED** might behave unexpectedly in a pivot table?

ALLSELECTED depends on the outer filter context created by slicers and visuals. In a pivot/matrix visual, row/column context and nested filters can produce an outer context you didn't expect, making ALLSELECTED return values that appear to ignore some filters. Also, drill-downs, expanded rows, and visual-level filters change the effective context.

10) Measure: Total Sales ignoring Region filters

DAX:

```
Total Sales (Ignore Region) = CALCULATE( SUM(Sales[Amount]), ALL(Sales[Region]) )
```

11) Optimize High Sales measure (replace FILTER with boolean filter)

DAX:

```
High Sales (Optimized) = CALCULATE( SUM(Sales[Amount]), Sales[Amount] > 1000 )
```

Explanation: Passing a boolean expression inside CALCULATE is evaluated as a row context converted to filter context and is more efficient than FILTER() which constructs a table.

12) Top 2 Products using TOPN and FILTER

DAX (table):

```
Top 2 Products Table = TOPN( 2, SUMMARIZE( Sales, Sales[ProductID], "ProductSales",  
SUM(Sales[Amount]) ), [ProductSales], DESC )
```

You can convert this into a measure or use it as a calculated table in visuals like a table visual.

13) ALLSELECTED with no parameters

ALLSELECTED() without parameters returns all rows from the current filter context (preserves slicer selection) but can ignore visual-level filters. Example usage in a percentage calculation: CALCULATE(SUM(Sales[Amount]), ALLSELECTED(Sales)).

14) Debugging SWITCH incorrect values in matrix

Common causes: - The measure relies on implicit row context (e.g., using raw column values instead of aggregator functions). - SELECTEDVALUE returns unexpected results when multiple rows exist in the context. - The measure is not using proper aggregation (e.g., SUM) and the matrix row expands context, changing results. Fixes: - Use aggregators like SUM, AVERAGE or wrap column references in SELECTEDVALUE with a default. - Use HASONEVALUE or ISINSCOPE to branch logic when a single item is present. - Evaluate intermediate variables with VAR and RETURN to make logic explicit.

15) Simulate a 'reset filters' button using ALL

Use a disconnected table as a slicer with an option 'All' and switch the measure to clear filters using ALL on selected columns. Example:

```
Sales with Toggle = VAR Mode = SELECTEDVALUE(ViewMode[Option], "Filtered") RETURN SWITCH( Mode,  
"All", CALCULATE(SUM(Sales[Amount]), ALL(Sales[Category])), SUM(Sales[Amount]) )
```

Sample Sales Data (as provided):

SaleID	ProductID	Amount	Region	SaleDate
1	P1	1200	North	1/5/2023
2	P2	800	South	1/10/2023
3	P1	1500	North	1/15/2023
4	P2	600	East	1/20/2023